Asynchronous operation
Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

March 16, 2025



Instructor: Thomas Feys

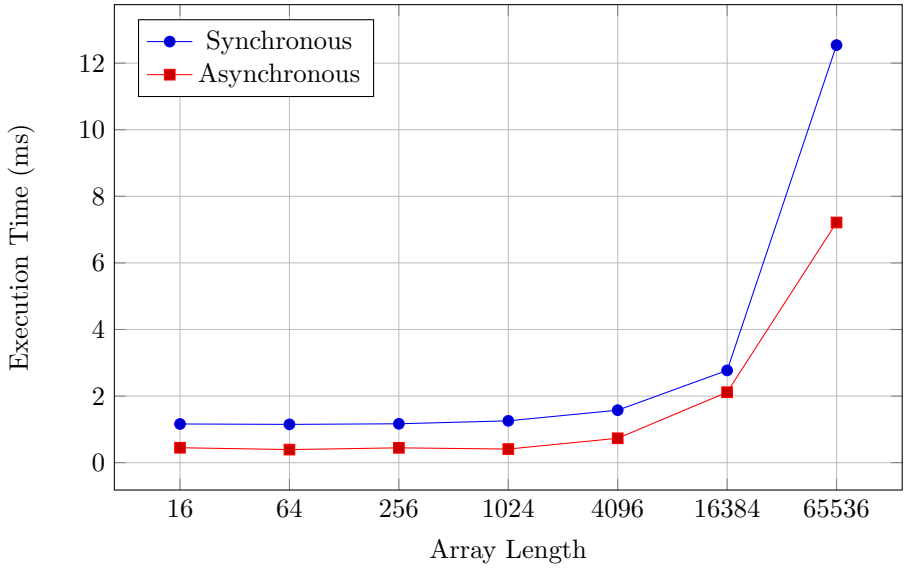
1 Introduction

This lab session covers asynchronous operation in GPU computing. Asynchronous execution reduces CPU-GPU idle time. We will explore their difference by examining their performance benefits.

2 Solution

From the results, we can conclude that the time difference between synchronous and asynchronous increases as the length of the array increases. This is expected because larger arrays require more computation and memory transfers, wich results in an improvement if we run asynchronous. This is due to the CPU idle time that is less in asynchronous mode.

For small arrays (size:16), we have an execution time of 1.162 and 0.449 for synchronous and asynchronous respectively. This is already a relatively large difference in execution time.



2.1 Code

2.1.1 Coalesced memory access

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <climits>

using namespace std;

// Kernel for different operations
__device__ int compute(int a, int b, int op) {
    switch (op) {
        case 0: return a + b;
        case 1: return a * b;
        case 2: return min(a, b);
        case 3: return max(a, b);
    }
    return a;
}

// Kernel reduction
__global__ void reductionKernel(int* d_arr, int* d_res, int N, int op) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < N) ? d_arr[idx] : (op == 1 ? 1 : INT_MIN);
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = compute(sdata[tid], sdata[tid + s], op);
        }
        __syncthreads();
    }

    if (tid == 0) {
        d_res[blockIdx.x] = sdata[0];
    }
}

// Final reduction kernel
__global__ void finalReduction(int* d_res, int* d_final, int numBlocks, int op) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    sdata[tid] = (tid < numBlocks) ? d_res[tid] : (op == 1 ? 1 : INT_MIN);
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = compute(sdata[tid], sdata[tid + s], op);
        }
        __syncthreads();
    }
    if (tid == 0) {
        *d_final = sdata[0];
    }
}
```

```
59
60 // Synchronous execution
61 void executeSync(int N, int blockSize) {
62     int ops[] = {0, 1, 2, 3};
63     int numBlocks = (N + blockSize - 1) / blockSize;
64     int* d_arr, * d_res, * d_final;
65     vector<int> h_arr(N);
66     int h_final;
67     cudaMalloc(&d_arr, N * sizeof(int));
68     cudaMalloc(&d_res, numBlocks * sizeof(int));
69     cudaMalloc(&d_final, sizeof(int));
70
71     auto start = chrono::steady_clock::now();
72     for (int op : ops) {
73         // new array
74         for (int j = 0; j < N; j++) h_arr[j] = rand() % 100 + 1;
75
76         cudaMemcpy(d_arr, h_arr.data(), N * sizeof(int), cudaMemcpyHostToDevice);
77         reductionKernel<<<numBlocks, blockSize, blockSize * sizeof(int)>>>(d_arr,
    d_res, N, op);
78         cudaDeviceSynchronize();
79         finalReduction<<<1, blockSize, blockSize * sizeof(int)>>>(d_res, d_final,
    numBlocks, op);
80         cudaDeviceSynchronize();
81         cudaMemcpy(&h_final, d_final, sizeof(int), cudaMemcpyDeviceToHost);
82     }
83     auto end = chrono::steady_clock::now();
84     cout << "Sync execution time: " << chrono::duration<double>(end - start).count
    () << " s" << endl;
85     cudaFree(d_arr); cudaFree(d_res); cudaFree(d_final);
86 }
87
88 // Asynchronous execution
89 void executeAsync(int N, int blockSize) {
90     int ops[] = {0, 1, 2, 3};
91     int numBlocks = (N + blockSize - 1) / blockSize;
92     int* d_arr[4], * d_res[4], * d_final[4];
93     vector<int> h_arr(N);
94     int h_final[4];
95     cudaStream_t streams[4];
96
97     for (int i = 0; i < 4; i++) {
98         cudaMalloc(&d_arr[i], N * sizeof(int));
99         cudaMalloc(&d_res[i], numBlocks * sizeof(int));
100        cudaMalloc(&d_final[i], sizeof(int));
101        cudaStreamCreate(&streams[i]);
102    }
103
104    auto start = chrono::steady_clock::now();
105    for (int i = 0; i < 4; i++) {
106        for (int j = 0; j < N; j++) h_arr[j] = rand() % 100 + 1;
107        cudaMemcpyAsync(d_arr[i], h_arr.data(), N * sizeof(int),
    cudaMemcpyHostToDevice, streams[i]);
108        reductionKernel<<<numBlocks, blockSize, blockSize * sizeof(int), streams[i
    ]>>>(d_arr[i], d_res[i], N, ops[i]);
109        finalReduction<<<1, blockSize, blockSize * sizeof(int), streams[i]>>>(
    d_res[i], d_final[i], numBlocks, ops[i]);
110        cudaMemcpyAsync(&h_final[i], d_final[i], sizeof(int),
    cudaMemcpyDeviceToHost, streams[i]);
111    }
112    cudaDeviceSynchronize();
113    auto end = chrono::steady_clock::now();
```

```
114     cout << "Async execution time: " << chrono::duration<double>(end - start).
        count() << " s" << endl;
115 
116     for (int i = 0; i < 4; i++) {
117         cudaStreamDestroy(streams[i]);
118         cudaFree(d_arr[i]); cudaFree(d_res[i]); cudaFree(d_final[i]);
119     }
120 }
121 
122 int main() {
123     const int N = 16;
124     const int blockSize = 1024;
125     executeSync(N, blockSize);
126     executeAsync(N, blockSize);
127     return 0;
128 }
```